Ders 23 Genişletilmiş Notu: Round Robin, Çok Seviyeli Kuyruklar, Çok İşlemcili ve Gerçek Zamanlı CPU Zamanlama

Ders 23 Genişletilmiş Notu: Round Robin, Çok Seviyeli Kuyruklar, Çok İşlemcili ve Gerçek Zamanlı CPU Zamanlama

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Önceki Dersten: Öncelikli Zamanlama ve Yaşlandırma
- 2. Round Robin Zamanlama
- 3. Çok Seviyeli Kuyruk Zamanlama
- 4. Çok Seviyeli Geri Beslemeli Kuyruk
- 5. İş Parçacığı Zamanlaması
- 6. Çok İşlemcili Zamanlama
- 7. Çok Çekirdekli İşlemciler ve Donanım İş Parçacıkları
- 8. Gerçek Zamanlı Sistemler ve Gecikmeler
- 9. Gerçek Zamanlı Zamanlama Algoritmaları
- 10. İşletim Sistemi Örnekleri: Linux ve Windows
- 11. Algoritma Değerlendirme
- Örnek Sorular

1. Önceki Dersten: Öncelikli Zamanlama ve Yaşlandırma

Ders, Silberschatz'ın *Operating System Concepts* kitabının 6. bölümünden (CPU zamanlama) devam etti. Hoca önce bir önceki derste anlatılan öncelikli zamanlamayı (priority scheduling) hatırlattı. Öncelikli zamanlama, bazı işleri öne alıp onların beklemesini en aza indirmek açısından güzeldir. Fakat yükün dağılımına bağlı olarak açlık (starvation) yaşanabilir: düşük öncelikli süreçler çok uzun süre bekletilebilir. Bu sorunu çözmek için yaşlandırma (aging) mekanizması kullanılıyordu. Bekleyen sürecin önceliği zamanla yükseltilir.

Bu derste konuşulacak round robin algoritması, bu yaklaşımdan biraz farklı bir algoritmadır.

2. Round Robin Zamanlama

2.1 Temel Fikir

Round robin (RR), yapı olarak ilk gelen ilk hizmet alır (first come first served, FCFS) algoritmasına benzer. Hoca burada sınıfa sordu: FCFS'de ne yok? Cevap: kesme (preemption) yok. RR'nin FCFS'den ilk farkı budur, RR'de preemption vardır. İkinci farkı amacıdır: RR, özellikle zaman paylaşımlı (time-sharing), yani etkileşimli (interactive) sistemler için geliştirilmiştir.

FCFS'de ilk geleni çalıştırırsınız, o bitince sıradakini çalıştırırsınız. Preemptive öncelikli zamanlamada ise düşük öncelikli bir süreç çalışırken yüksek öncelikli bir süreç gelirse doğrudan yüksek öncelikliyi çalıştırırsınız. Her iki durumda da süreçler arasında fazladan bekleme oluşur. RR bu beklemeyi azaltmak, etkileşimli bir sistemde daha etkin çalışmak için ortaya atılmıştır.

RR'de işlemci zamanı, zaman dilimi (time quantum) denen parçalara bölünür. Hazır kuyruğundaki (ready queue) her süreç sırayla alınır ve CPU'da en fazla bir time quantum kadar çalıştırılır. Süreç hâlâ çalışmak istiyorsa kuyruğun sonuna konur. Sürecin burst'ü quantum'dan kısaysa süreç quantum dolmadan biter. Bu durumda quantum'un dolması beklenmez, kuyruktan başka bir süreç alınıp çalıştırılır. Dolayısıyla hazır kuyruğu dairesel (circular) bir kuyruktur.

Hoca, RR'nin bugünkü işletim sistemlerinde burada anlatılan saf hâliyle olmasa da başka yapılarla birlikte kullanıldığını söyledi. Örnek olarak Linux'taki "completely fair" zamanlayıcıyı ve Windows'taki, round robin destekli çok seviyeli geri beslemeli kuyruk yapısını verdi.

> **Ders sonrası not:** Hoca Linux için "complete fair queuing" dedi. Linux'un CPU zamanlayıcısının adı Completely Fair Scheduler'dır (CFS). CFQ (Completely Fair Queuing) ise Linux'ta bir disk G/Ç zamanlayıcısıdır.

Buradaki en önemli nokta şudur: Seçilen time quantum değeri sistemin başarımını doğrudan etkiler.

2.2 Örnek: 24, 3, 3 ve $q = 4$

Hoca, FCFS'den hatırlanan "24-3-3" örneğini kullandı. P1, P2, P3 süreçleri bu sırayla gelir. Burst süreleri 24, 3 ve 3'tür. FCFS'de bekleme süreleri P1 için 0, P2 için 24, P3 için 27 idi. Toplam 51, ortalama 17 birimdi. P2 ve P3 uzun süre hiçbir sonuç üretemiyordu.

Şimdi aynı süreçleri RR ile çalıştıralım. Sistem genelinde tek bir quantum vardır: $q = 4$.

- 0-4: P1 çalışır. 24 birimlik burst'ün 4 birimini kullanır, kalan 20. P2 ve P3'ün arkasına konur.
- 4-7: P2 çalışır. Burst'ü 3, quantum'dan küçük. Quantum dolmadan biter.
- 7-10: P3 çalışır, 3 birimde biter.
- 10-30: Kuyrukta yalnız P1 kalmıştır. Her quantum sonunda yine P1 seçilir: 10-14, 14-18, 18-22, 22-26, 26-30.

Gantt sırası: P1, P2, P3, P1, P1, P1, P1, P1.

Bekleme süresi, bitiş zamanı eksi burst süresidir (hepsi 0 anında geldiği için) (ara adım eklendi):

Süreç	Burst	Bitiş	Bekleme
P1	24	30	$30 - 24 = 6$
P2	3	7	$7 - 3 = 4$
P3	3	10	$10 - 3 = 7$

Ortalama bekleme: $(6 + 4 + 7)/3 = 17/3 \approx 5{,}67$ birim. FCFS'deki 17 birimle karşılaştırınca büyük bir iyileşmedir. Sonuçlar Python ile doğrulandı.

2.3 Time Quantum Seçimi

Quantum küçükse: Bağlam değişimi (context switch) sayısı artar. Hoca bunu $q = 1$ ile anlattı. P1 bir birim çalışır, P2'ye geçilir, bir birim çalışır, P3'e geçilir, bir birim çalışır. Sıra P1, P2, P3, P1, P2, P3 diye gider. Sonra hep P1 çalışır. $q = 4$'teki şemada hoca 7 geçiş saydı. $q = 1$'de bu sayının "20 küsürlere fırlayacağını" söyledi.

> **Ders sonrası not:** Python ile sayıldı. $q = 4$'te 8 dilim, yani 7 dilim sınırı vardır. Bunların yalnız 3'ü farklı süreçler arasındadır. Geri kalan 4'ünde P1 yine P1'e geçer. $q = 1$'de 30 dilim, 29 dilim sınırı vardır. Farklı süreçler arası geçiş 9'dur. Hocanın saydığı sayılar dilim sınırlarıdır.

Bağlam değişimi istenen bir şey değildir. Her biri küçük de olsa, sürekli bağlam değişimi yapılırsa üst üste biriken bir ek yük (overhead) oluşur. Bu yüzden time quantum, bağlam değişimine harcanan zamana göre büyük olmalıdır.

Quantum büyükse: Etkileşimi sağlamak istiyorsak quantum çok büyük de olmamalıdır. Çok uzun tutulursa RR, FCFS'ye döner. Bu örnekte en büyük değeri, 24'ü verirsek P1 baştan sona

çalışır, P2 ve P3 bekler. Python ile bakıldığında $q = 24$ için ortalama bekleme 17'dir, yani FCFS ile aynıdır.

> **Hocanın vurgusu:** Time quantum değerinin seçimi kritiktir. Quantum büyüdükçe bağlam değişimi azalır. Ama FCFS'den farkı kalmaz ve bekleme süresi (waiting time) artar.

Hoca bugünkü sistemlerde bu değerin slayttaki 10-100 ms aralığında olduğunu söyledi. Windows tarafında yanlış hatırlamıyorsa 20 ms civarında, Linux'ta ise (zamanlayıcısı daha karmaşık olsa da) 100 ms civarındadır. Bu değer sistemin çalışma hızına ve başarımına, ayrıca tek işlemcili ya da çok işlemcili olmasına bağlı olarak da değişir.

2.4 Turnaround Süresi ve Quantum İlişkisi

Slaytta dört süreçlik bir küme için bir grafik vardı. P1, P2, P3, P4 süreçlerinin burst'leri 6, 3, 1 ve 7'dir. Grafik, quantum değerine göre ortalama dönüş süresini (turnaround time) gösterir. Hoca grafikteki değerleri okudu. Değerler Python ile hesaplandı (hepsi 0 anında, P1-P4 sırasıyla geliyor):

q	1	2	3	4	5	6	7
Ort. turnaround	11,0	11,5	10,75	11,5	12,25	10,5	10,5
Ort. bekleme	6,75	7,25	6,5	7,25	8,0	6,25	6,25

> **Ders sonrası not:** Hoca $q = 3$ için "11'in altına düşüyor", $q = 5$ için "12'ye çıkıyor" dedi. Hesaplanan değerler 10,75 ve 12,25'tir. Bekleme satırı derste verilmedi, Python ile eklendi.

Bu tablo, az önce söylenenle çelişiyor gibi görünebilir: turnaround, quantum büyüdükçe düzenli biçimde kötüleşmiyor. Quantum en uzun süreç süresine yaklaşınca turnaround düşüyor. Hoca burada iki farklı ölçütten söz edildiğini vurguladı. Grafikte yalnız turnaround'a bakılıyor. Turnaround'un uzaması, sürecin sistemde daha uzun kalması, çalışma süresinden daha uzun sürede tamamlanması demektir.

> **Hocanın vurgusu:** Bu grafiğe bakarken yalnız turnaround time'a baktığımızı unutmayın. Derdimiz tepki veren (responsive) bir sistem üretmekse asıl bakacağımız şey turnaround değil, bekleme süresidir.

Hoca bir genel kural da verdi: Quantum, süreçlerin %80'inin burst'ünden uzun seçilirse turnaround makul seviyelerde kalır. Örnekte burst'ler küçükten büyüğe 1, 3, 6, 7'dir. Hoca en kısa üçünü sıralayıp $(1, 3, 6)$ quantum'u 6 seçmenin turnaround'u etkin hâle getirdiğini söyledi. Ama bekleme süresi açısından ayrıca incelenmesi gerekir.

> **Ders sonrası not:** Slayttaki kural "CPU burst'lerinin %80'i q'dan kısa olmalı" biçimindedir. Hocanın anlatımıyla aynı anlama gelir. Örnekte $q = 6$ seçilince dört burst'ün üçü (%75) quantum'a sığar. Hoca bu oranı "%83" diye söyledi gibi duyuluyor.

3. Çok Seviyeli Kuyruk Zamanlama

Bir başka yaklaşım, sistemde birden fazla kuyruk oluşturmak ve bu kuyruklara ayrı ayrı, farklı şekillerde davranmaktır. Bu yöntem, süreçleri farklı karakterde gruplara ayırabildiğimizde kullanılır.

Slayttaki basit örnek, süreçleri ön plan (foreground) ve arka plan (background) diye ikiye bölmektir. Bir masaüstü bilgisayarda kullanıcının etkileşime girdiği uygulamalar ön plandadır, etkileşimlidir. Sistemin çalışması için arkada yapılması gereken uzun süreli, toplu (batch) karakterdeki işler ise arka plandadır. Önce ön plandaki işler tamamlanmaya çalışılır, kalan zamanda arka plan işleri yapılır.

Hoca Windows'tan bir örnek verdi: Windows da ön plandaki süreçlerden kalan zamanda arka planda kötü amaçlı yazılım taraması, disk taraması gibi işleri çalıştırmaya çalışır. Ama hocaya göre bu çoğu zaman becerilemiyor. Arka plan süreçleri bazen ön plandaki çalışmayı neredeyse engelleyecek noktaya geliyor.

Çok seviyeli kuyrukta (multilevel queue) bir süreç hangi kuyrukta başlıyorsa hayatına o kuyrukta devam eder. Her kuyruk farklı bir zamanlama algoritması kullanabilir:

- Ön plan kuyruğunda RR: Etkileşimli süreçler çalışacağı için herkesin biraz biraz ilerlemesi, zaman paylaşımlı bir yapı gerekir.
- Arka plan kuyruğunda FCFS: Yapılacak işler sıraya konur, sırayla gider.

Kuyrukların kendi aralarında da bir zamanlama yöntemi belirlenmelidir. Birkaç seçenek vardır:

- **Sabit kural:** Arka plandaki bir işin çalışması için ön plan kuyruğunda hiç iş kalmamış olmalıdır.
- **Oransal paylaşım:** CPU kullanımı iki kuyruk arasında belirli bir oranda dengelenir. Örneğin ön plan işleri CPU zamanının %80'ini, arka plan işleri %20'sini alır.

Kuyruk sayısı ve kuyruklarda kullanılan algoritmalar konusunda hiçbir sınır yoktur. Genel adı çok seviyeli kuyruk zamanlamadır (multilevel queue scheduling). Kuyruk sayısını ya da kullanılan algoritmayı değiştirdiğinizde her seferinde yeni bir yöntem oluşturmuş olursunuz.

4. Çok Seviyeli Geri Beslemeli Kuyruk

Çok seviyeli kuyruk yapısını daha dinamik hâle getirelim. Bir geri bildirim mekanizması kurar ve buna göre süreçlerin kuyruklar arasında ilerlemesine izin verirsek, yani bir kuyrukta başlayan süreç sonra başka bir kuyruğa, sonra bambaşka bir kuyruğa geçebiliyorsa, çok seviyeli geri beslemeli kuyruktan (multilevel feedback queue) söz ederiz.

Bu yapıyı tanımlayan parametreler şunlardır:

- Kuyruk sayısı.
- Her kuyrukta kullanılan zamanlama algoritması.
- Bir sürecin ne zaman üst kuyruğa yükseltileceği (promote).
- Bir sürecin ne zaman alt kuyruğa indirileceği (demote).
- Yeni oluşturulan sürecin hangi kuyruktan başlayacağı.

Hoca bunların toplamda dört, hatta üç başlığa da indirilebileceğini söyledi. Çünkü yükseltme, indirme ve başlangıç kuyruğu benzer kararlardır, tek başlıkta toplanabilir. Bunlar her farklı şekilde tanımlandığında yeni bir çok seviyeli geri beslemeli kuyruk elde edilir.

4.1 Kriter: CPU Kullanımı

En kısa iş önce (shortest job first, SJF) algoritmasından hatırlanacağı gibi, kısa işleri öne geçirip bir an önce bitirmek bekleme süresini azaltır. Bu yüzden bir kriter olarak sürecin ne kadar CPU zamanı kullandığına bakılabilir.

Çok fazla CPU zamanı kullanan bir süreç bir alt kuyruğa indirilir. Daha da fazla kullanıyorsa bir alt kuyruğa daha indirilir. Her indirmede şunu söylemiş oluruz: Süreç kendisine verilen dilimin tamamını kullandı ve hâlâ bitmedi. O hâlde daha fazla kullanabileceği bir yere gitsin. Alt kuyruklarda verilen zaman dilimi artırılır. Fakat asıl istenen kısa işlerin önce çalışmasıdır. Bu yüzden kuyruklar arasında ters bir öncelik yapısı kurulur: Üstteki kuyruk daha önceliklidir.

4.2 Slayttaki Üç Kuyruklu Örnek

- Q_0 (en üst): RR, $q = 8$.
- Q_1: RR, $q = 16$.
- Q_2 (en alt): FCFS. Hocanın deyişiyle burada süreç "bodoslama", bitene kadar çalışır.

İşleyiş şöyledir:

- Yeni gelen süreç doğrudan Q_0'a girer ve RR içinde 8 birimlik quantum alır.
- 8 birimde işini bitiremezse, yani hâlâ CPU kullanma isteği varsa, Q_1'in sonuna atılır. Burada quantum 16'dır.
- Burada da quantum'u kullandıktan sonra hâlâ çalışması gerekiyorsa Q_2'nin sonuna atılır. Orada kendisi gibi bekleyen işlerin sonuna gelir.

Her seferinde süre artırılıyor gibi görünür. Ama bir de şu kural vardır: Q_1'de bir işin zamanlanabilmesi için Q_0'da zamanlanacak bir iş olmamalıdır. Benzer şekilde Q_2'deki bir işin zamanlanabilmesi için Q_0 ve Q_1 boş olmalıdır. Önceliklendirme bu şekilde yapılır.

5. İş Parçacığı Zamanlaması

Hoca, tek CPU üzerindeki temel zamanlama algoritmalarının bunlar olduğunu söyledi. Gerçek zamanlı CPU zamanlama için başka algoritmalar da ileride konuşulacaktı. Ama önce 4. bölümden bir şey hatırlattı: kullanıcı düzeyi (user level) ve çekirdek düzeyi (kernel level) iş parçacıkları (thread).

İşletim sistemi çekirdek thread'lerini doğrudan destekliyorsa zaten süreçlerin değil, thread'lerin zamanlanmasından söz ederiz. Kullanıcı düzeyi thread'ler ise genelde bir thread kütüphanesi (thread library) tarafından gerçeklenir. Çekirdek çoğu zaman bunlardan habersizdir. Ama arada bir eşleştirme yapılması gerekir. Bu genelde hafif süreç (lightweight process, LWP) üzerinden olur.

5.1 Süreç Çekişme Kapsamı ve Sistem Çekişme Kapsamı

Kullanıcı thread'leriyle çekirdek thread'lerini eşleştiren modeller, hibrit sayılmazsa, üç taneydi: many-to-one, many-to-many ve one-to-one.

Many-to-one ve many-to-many modellerinde aradaki thread kütüphanesi, kullanıcı düzeyi thread'lerin LWP'ler üzerinde nasıl zamanlanacağına karar verir. Buna süreç çekişme kapsamı (process contention scope, PCS) denir. Çünkü aynı sürece bağlı thread'lerin kendi aralarında yarışmasından söz ediyoruz.

Ama dikkat: Arada bir aracı (broker) vardır, LWP. Thread'ler aslında LWP için yarışır. LWP'yi kazanmaları, o anda CPU'da çalıştıkları anlamına gelmez. Bunun için çekirdek thread'inin de çekirdek zamanlayıcısı tarafından CPU'ya zamanlanması gerekir. Bir çekirdek thread'inin CPU'da zamanlanması için de sistemdeki bütün çekirdek thread'leri arasında bir yarışma olur. Buna sistem çekişme kapsamı (system contention scope, SCS) denir.

One-to-one modelini kullanan işletim sistemleri, örneğin Linux ve Windows, thread zamanlamasını doğrudan SCS üzerinden yapar. Hocaya göre bu makul ve mantıklıdır.

5.2 Pthreads'te Kapsam

POSIX thread kütüphanesinde (Pthreads) zamanlama için thread özniteliklerinin (attribute) tutulduğu bir yapı vardır. Bunun iki fonksiyonu vardır: `pthread_attr_getscope` ve `pthread_attr_setscope`. Bunlarla zamanlamanın hangi kapsamda yapılacağı belirlenebilir. Ama bu, üzerinde çalışılan işletim sistemi tarafından sınırlanabilir. Örneğin Linux ve macOS'ta yalnız sistem kapsamı (`PTHREAD_SCOPE_SYSTEM`) çalışır, süreç kapsamında zamanlama yapılamaz.

Slayttaki örnek kod yalnız bir öznitelik tanımlar, onu başlatır, sonra kapsamını alıp ekrana yazar. Hoca bunun üzerine fazla düşünmeye gerek olmadığını söyledi.

6. Çok İşlemcili Zamanlama

Şu ana kadar hep tek işlemcili bir sistemde zamanlamanın nasıl olacağı konuşuldu. Çok işlemcili zamanlamada (multiprocessor scheduling) iş biraz karmaşıklaşır. Temel problem yine aynıdır: CPU zamanını iyi kullanmak. Değişen birkaç şey vardır. İlki mimaridir.

6.1 Asimetrik ve Simetrik Çok İşlemcilik

Zamanlama işleminde yapılan şey, bir kuyruktan bir süreç seçip boştaki bir işlemciye atamaktır. Birden fazla işlemci ortak bir kuyruk üzerinden bunu aynı anda yapmaya kalkarsa, çekirdekteki ortak veri yapılarının korunması gerekir. Hoca burada yakın zamanda konuşulan senkronizasyonu hatırlattı.

- **Asimetrik çok işlemcilik (asymmetric multiprocessing):** Bütün sistem tek bir merkezden yönetilir, zamanlamayı tek bir işlemci yapar. Veri yapılarına yalnız bir işlemci eriştiği için senkronizasyon derdi yoktur. İşler çok daha rahat yürür.
- **Simetrik çok işlemcilik (symmetric multiprocessing, SMP):** Her işlemci kendi zamanlamasını yapar. Burada da seçenekler vardır. Ortak bir hazır kuyruğu kullanılırsa, birden fazla işlemcinin aynı kuyruğa erişmeye çalışması senkronizasyon problemi getirir. Bunu çözmenin en temel yolu her işlemciye ayrı bir kuyruk vermektir.

Hoca sınıfa sordu: Ayrı kuyruk bütün sorunları çözer mi? Cevap: Çözmez. Her işlemci için ayrı kuyruk oluşturduğumuzda bile sistemdeki bütün işlemcilerin etkin biçimde kullanıldığından emin olmamız gerekir. Bir kuyruk ağzına kadar dolu, herkes bekliyor; diğeri bomboş. Bu kötü bir durumdur ve dengelenmesi gerekir. Ama dengelerken dikkat edilmesi gereken bir nokta vardır: işlemci yakınlığı.

6.2 İşlemci Yakınlığı

Bir süreç işlemcide çalışırken bellek erişimi yaptıkça, sonraki erişimleri hızlandırmak için ilgili bilgiyi önbelleğe (cache) koyarız. Böylece bazı bellek erişimleri doğrudan önbellekten, çok daha hızlı yapılır.

Şimdi bu süreç çalışıyordu, quantum'u doldu ve kenara alındı. Bir sonraki sefer onu CPU 0 yerine CPU 1'e verdiğimizi düşünelim. Ne kazanır, ne kaybederiz?

- CPU 0'ın önbelleğindeki bilgiler artık işe yaramaz, çünkü orada o süreç çalışmıyor. Bu bilgiler silinmez ama geçersiz (invalid) hâle gelir, üzerine yazılır.
- CPU 1'in önbelleğinde bu sürecin eriştiği bellek adresleriyle ilgili hiçbir şey yoktur. Önbelleğin yeniden doldurulması gerekir. Süreç ilk bellek erişimini yaptığında bilgi tekrar önbelleğe çekilir.

Bu yüzden SMP sistemlerinde genellikle (hoca "hep" demenin fazla iddialı olacağını belirtti) bir işlemcide zamanlanan sürecin hep aynı işlemcide zamanlanması sağlanmaya çalışılır. Buna işlemci yakınlığı (processor affinity) denir.

Hoca bunu yumuşak ve sert son tarihlere benzetti. İki türü vardır:

- **Yumuşak yakınlık (soft affinity):** İşletim sistemi "Bir süreci bir işlemcide zamanladıysam bundan sonra hep orada zamanlamaya çalışacağım, ama garanti vermiyorum" der.
- **Sert yakınlık (hard affinity):** Bir sürecin çalışabileceği işlemciler sınırlanır: "Bu süreç yalnız şu ve şu işlemcide çalışabilir."

Sınırlama genellikle tek bir işlemciyle değil, bir işlemci grubuyla yapılır. Çünkü önbellekler seviyeli (L2, L3 vb.) olduğundan farklı işlemciler aynı önbelleğe erişebilir. Bu da kısıtlamayı esnetir.

6.3 NUMA

Yakınlığı biçimlendiren kısıtlardan biri de sistemin ana bellek mimarisidir. Hocaya göre bu, kullandığımız masaüstü bilgisayarlarda pek görülmez. Ama özellikle 16, 32, 64, 100, 500 gibi sayılarda işlemciye sahip olabilen sunucu mimarilerinde, hatta iki işlemcili sistemlerde bile şunu görürsünüz: Bazı bellek yuvaları (slot) bir işlemciye, bazıları diğer işlemciye atanmış gibidir. Bu yüzden bellek takarken yuvalar 0, 1, 2, 3 diye değil, 0, 4, 8, 1, 5, 9 gibi farklı dizilimlerle doldurulur.

Bu sistemler çoğunlukla düzgün olmayan bellek erişimlidir (non-uniform memory access, NUMA). Her işlemcinin doğrudan ve hızlı eriştiği tanımlı bir bellek bölgesi vardır. Diğer bölgelere erişim dolaylı olduğu için daha yavaştır. Sistem NUMA ise yakınlık daha da önemli hâle gelir: Sürecin eriştiği bellek bölgesi ana bellekte nereye denk düşüyorsa, o bölgeye yakın işlemcide ya da işlemci grubunda çalışması daha mantıklıdır.

6.4 Yük Dengeleme

Kuyruklar ayrı olduğunda dengesizlik ihtimali vardır. Bir süreç atanırken çoğu zaman burst'ünün ne olacağı, ne kadar G/Ç yapacağı bilinmez. Bu yüzden SMP bir sistemde farklı işlemcilerin hazır kuyruklarında dengesizlik oluşabilir. Yükü düzgün dağıtmaya yük dengeleme

(load balancing) denir. İki temel yöntem vardır ve SMP çalışan bugünkü işletim sistemleri ikisini de destekler:

- **İtme ile taşıma (push migration):** Yük dengeleme için ayrı, özel bir görev (task) vardır. Bu görev periyodik olarak yükleri kontrol eder. Bir dengesizlik görürse, yoğun kuyruktaki süreçleri daha boş kuyruğa taşır. Dengesizlik için tanımlı bir eşik (threshold) vardır; bir kuyrukta bir iş fazla diye taşıma yapılmaz.
- **Çekme ile taşıma (pull migration):** Boştaki, kuyruğunda bir şey olmayan işlemci, yüklü bir işlemcinin kuyruğunda bekleyen işlerden kendi kuyruğuna çeker ve çalıştırmaya başlar.

Bu iki yöntem birbirini dışlamaz. Hoca Linux zamanlayıcısında ikisinin birden gerçeklendiğini söyledi.

> **Hocanın vurgusu:** Her iki yöntemin de bir ek yükü (overhead) vardır, bunu unutmayın. Yalnız işlemci zamanı açısından bakınca taşıma kârlı görünür. Ama önbellek etkisi yüzünden zaman zaman zararlıdır ve bu işi yapmak da ayrıca bir yük olduğu için getirisi götürüsünden az olabilir.

7. Çok Çekirdekli İşlemciler ve Donanım İş Parçacıkları

Bugün SMP sistem denince her zaman birden fazla fiziksel işlemci entegresi kastedilmez. Bazen tek bir entegrenin içinde birden fazla işlemci (çekirdek) bulunur. Bu, enerji tüketimi ve yer açısından daha etkindir. Ama zamanlamayı biraz daha karmaşık hâle getirir.

Hesaplama yaparken bellek erişimine ihtiyaç duyarız. Bilgi önbellekte değilse bellek erişiminde zaman kaybedilir. Belleğin çalışma hızı işlemcinin çalışma hızıyla bir değildir, arada beklemeler olur. Buna bellek beklemesi (memory stall) denir. Slayttaki grafikte bir hesaplama (compute cycle) başlar, araya bir bellek erişimi girer ve işlemci bekler. Hoca, bu iki sürenin eşit olduğu düşünülürse işlemcinin zamanının %50'sini beklemekle geçirdiğini söyledi.

Bu durum avantaja çevrilebilir: İşlemcinin içinde iki ayrı donanım thread'i (hardware thread) tanımlanır. Birinin hesaplama döngüsü, diğerinin bellek bekleme döngüsüyle üst üste bindirilir. Böylece CPU zamanı daha etkin kullanılır. Oran mimariden mimariye değişebilir. Oran farklı olunca donanım thread sayısı da artabilir; ikiden aşağı zaten inemez.

Her donanım thread'i, yukarıdan, işletim sistemi tarafından bakıldığında ayrı bir işlemci gibi görünür. Hoca, Windows'ta görülen işlemci sayısının ve "sanal thread" sayısının buradan geldiğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Bir çekirdeğin birden fazla donanım thread'ini çalıştırmasına genel olarak çok iş parçacıklı çekirdek (multithreaded core) ya da eşzamanlı çoklu iş parçacığı (simultaneous multithreading, SMT) denir. Intel bu teknolojiyi Hyper-Threading adıyla pazarlar. İşletim sisteminin gördüğü mantıksal işlemci sayısı, büyük olasılıkla fiziksel çekirdek sayısı çarpı çekirdek başına donanım thread sayısıdır.

8. Gerçek Zamanlı Sistemler ve Gecikmeler

8.1 Gerçek Zamanlı Sistem Nedir?

Zamanlamanın biraz özel bir noktası gerçek zamanlı CPU zamanlamadır (real-time CPU scheduling). Hoca önce gerçek zamanlı sistemin ne olduğunu sordu. Ona göre "Gerçek zamanlı zamanlama nedir?" sorusuna bazen "gerçek zamanlı çizelgeleme" diye cevap veriliyor ve bu kâbus gibi bir cevap.

Soru: Gerçek zamanlı sistem nedir? **Cevap (bir öğrenci):** Önceden belirlenmiş bir zaman dahilinde görevlerin gerçekleşmesini garantileyen işletim sistemleri. **Hocanın düzeltmesi:** Hem doğru hem yanlış. Önceden belirlenmiş bir çizelge olmak zorunda değil. Zamanlama çevrimdışı (offline) yapılıp sistem buna göre planlanabilir, ya da çalışırken (online) yapılabilir. Gerçek zamanlı sistem aslında zamana duyarlı (time-sensitive) bir sistemdir. Amaç, sistemdeki görevlerin belirlenen süre içinde tamamlanabilmesidir.

8.2 Yumuşak ve Sert Gerçek Zamanlı Sistemler

Hoca evdeki uydu alıcısını örnek verdi. Kumandaya bastığınızda kanalı değiştirmesi, sesi artırması ya da bir menüye geçmesi gerekir. Bunu belli bir sürede yapmasını istersiniz. Düğmeye bastıktan sonra bir sonraki kanala yarın, bir dakika sonra ya da otuz saniye sonra geçmesinin anlamı yoktur. Kabul edilebilir süre örneğin 0,5 saniyedir. Bu süre içinde tamamlanınca kimse madalya takmaz. 0,5 yerine 0,6 saniyede tamamlanınca da kimse uydu alıcısını kırmaz, başınıza ciddi bir şey gelmez. Bu bir yumuşak gerçek zamanlı (soft real-time) sistemdir: Bir olaya ya da periyodik bir işe karşılık yapılması gereken işler belli bir süre içinde bitmelidir, ama süre aşıldığında ölümcül ya da ciddi bir sorun çıkmaz.

Sert gerçek zamanlı (hard real-time) bir sistemde ise bir işin son tarihini (deadline) kaçırması, o sistemin, onu kullanan sistemlerin ya da kişilerin başına ciddi dert açması demektir. Hocanın örneği araçtaki ABS sistemidir. ABS belli bir süre içinde arabanın kaydığını anlamalı, buna karşılık freni belli bir periyotla sıkıp gevşetmelidir. Kaydığını geç anlarsa kaza yaparsınız. Kaydığını anladığı hâlde, 5 milisaniyelik “fren yap, aç” döngüsü yerine 10 milisaniyede bir kayarsa yine sıkıntı olur, yine kaza yaparsınız.

> **Hocanın vurgusu:** Bir sistemin sert ya da yumuşak gerçek zamanlı olmasını belirleyen şey, bir son tarihi kaçırdığınızda başınıza ne geldiğidir. Hangisi olursa olsun, gerçek zamanlı sistem işlerini belli bir zaman sınırı içinde bitirmelidir.

8.3 Kesme Gecikmesi ve Dağıtım Gecikmesi

Bu zaman sınırını etkileyen iki durum vardır.

Kesme gecikmesi (interrupt latency): Bir kesme, yani bir olay oluşur. Bu yazılımsal ya da donanımsal olabilir. Kesmenin oluştuğunun anlaşılması, tipinin belirlenmesi, çalışmakta olan sürecin kenara alınması ve bu kesmeyle ilgili servis rutininin (interrupt service routine) başlatılması gerekir. Bunlar için geçen süreye kesme gecikmesi denir.

Dağıtım gecikmesi (dispatch latency): Çalışmakta olan sürecin işlemciden alınıp diğerinin işlemciye konması için geçen süredir. Slayttaki ayrıntılı grafikte bir olay oluşur, o olaya karşı yapılması gereken iş anlaşılır, ilgili süreç ya da servis rutini işlemciye konur, çalışır ve biter. Buraya kadar geçen her sürenin bir tanımı vardır. Dağıtım gecikmesi, CPU’da çalışacak işin değiştirilmesi kısmıdır.

Bu kısımda çözülmesi gereken birden fazla nokta olabilir. Bunlardan biri çakışmaların (conflict) çözülmesidir. Çalıştıracağınız işle ilgili bir kaynak, o anda çalışmayan bir süreç tarafından tutuluyor olabilir. Sistemin çalışması için bu kaynağın yeniden kullanılabilir hâle getirilmesi gerekebilir. Bu çakışma kısmı dağıtım gecikmesinin içindeki ayrı bir parçadır.

9. Gerçek Zamanlı Zamanlama Algoritmaları

9.1 Öncelik ve Periyodik Görevler

Gerçek zamanlı sistemlerde hep bir önceliklendirme üzerinden gitmek gerekir. Çünkü oluşan olaylara servis vermek için kısıtlı süreler vardır. Kumandada düğmeye basıldıysa geçişi şu kadar sürede bitirmek gerekir. Bir MP3 çalarda stop tuşuna basıldıysa çalma durdurulmalıdır. Stop’a bastıktan iki saniye sonra durdurmak makul değildir. Play’e basıldığında da benzer şekilde belli bir sürede cevap verilmelidir.

Gerçek zamanlı sistemler zaten belli bir iş için özel olarak hazırlanmış sistemlerdir. Bu yüzden ortaya çıkan işler çoğunlukla periyodiktir. Bir kısmı periyodik değildir; bir olayın ne zaman oluşacağını önceden bilemezsiniz. Ama oluştuğunda çalıştırılması gereken işin ne olduğunu, aşağı yukarı ne kadar süreceğini ve ne kadar sürede tamamlanması gerektiğini bilirsiniz. Bu bilgilerle önceliklendirilmiş zamanlama yöntemleri kurulabilir.

Periyodik bir görevin üç değeri vardır:

- **Periyot (p):** Görevin hangi aralıkla tekrarlandığı.
- **Son tarih (d):** Görevin ne kadar sürede tamamlanması gerektiği.
- **Çalışma süresi (t):** İşin normal şartlarda ne kadar sürdüğü. Hoca bunun başka kaynaklarda e ile gösterildiğini de söyledi.

Hocanın örneği gömülü bir sistemdeki ekrandır. Ekranın saniyede 25 kez, yani 40 milisaniyede bir yenilenmesi gerekir: $p = 40$ ms. Yenilemenin süresi örneğin 5 milisaniyeden fazla olmamalıdır: $d = 5$ ms. Yenileme işinin kendisi belki 1 milisaniye sürer: $t = 1$ ms.

Sistemin zamanlanabilmesi için normalde

$$0 \leq t \leq d \leq p$$

olmalıdır. Çalışma süresi son tarihten büyükse son tarih hiçbir zaman yakalanamaz.

9.2 Hız Monoton Zamanlama

Bu bilgiler ışığında farklı algoritmalar ortaya atılmıştır. Birincisi hız monoton zamanlamadır (rate monotonic scheduling). Öncelik değeri periyotla ters orantılıdır: Periyot kısaysa öncelik yüksektir. Biri 20 milisaniyede bir, diğeri 5 milisaniyede bir ortaya çıkan iki iş varsa, 5 milisaniyede bir ortaya çıkan iş daha önceliklidir. Sistemde düşük öncelikli bir süreç çalışırken yüksek öncelikli bir süreç (periyodu geldiği için) hazır olursa, düşük öncelikli kesilir ve yüksek öncelikli çalıştırılır.

Örnek 1: P1 için $p_1 = 50$, $t_1 = 20$. P2 için $p_2 = 100$, $t_2 = 35$. Son tarih periyoda eşittir. İkisi de 0 anında sistemdedir. P1'in periyodu daha kısa olduğu için önceliği yüksektir.

- 0-20: P1 çalışır ve biter.
- 20-50: P2 çalışır. Normalde 35 birim çalışıp 55'te bitmesi gerekirdi. Ama 50'de P1 yeniden ortaya çıkar ve P2 kesilir. P2'nin 30 birimi tamamlanmıştır, 5 birimi kalmıştır.
- 50-70: P1 çalışır.
- 70-75: P2 kalan 5 birimi çalışır ve biter. İlk son tarihi 100'dür, yakalanır.
- 75-100: CPU boştur.
- 100'de iki süreç yeniden gelir ve aynı desen tekrarlanır: 100-120 P1, 120-150 P2, 150'de P1 tekrar gelir, 150-170 P1, 170-175 P2.

Hiçbir son tarih kaçırılmaz. Bu, CPU kullanımıyla da kontrol edilebilir. Burst'ü periyoda bölerseniz:

$$U = \frac{20}{50} + \frac{35}{100} = 0{,}40 + 0{,}35 = 0{,}75$$

Toplam 1'den küçük olduğu için bunu zamanlayabiliriz diye düşünebilirsiniz. Hoca "diye düşünebilirsiniz" dediğinin altını çizdi: Her zaman olmaz.

Rate monotonic ne zaman işe yaramaz? Hoca bunu sınıfa sordu (ayrıntısı Örnek Sorular bölümünde). Bir öğrenci, uzun periyotlu işin daha önemli olabileceğini söyledi. Hoca "önemli" ve "kritik" kavramlarının tanımlanması gerektiğini, neye göre ve kime göre önemli olduğunu sordu. Asıl cevabı son tarihlerle verdi.

Görevlerin üç zamanı vardır: periyot, son tarih, çalışma süresi. Bu sırayla büyükten küçüğe giderler. Önceki örnekte son tarihten hiç söz edilmedi, son tarih periyoda eşit kabul edildi: İşi bir sonraki periyot gelmeden bitirmek yeterliydi. Son tarih periyottan kısa da olabilir, eşit de olabilir. Ama her iki durumda da rate monotonic yalnız hıza (periyoda) bakarak zamanlama yapar. Slayttaki başlık da buydu: rate monotonic zamanlamada kaçırılan son tarihler (missed deadlines).

Örnek 2: P1 için $p_1 = 50$, $t_1 = 25$. P2 için $p_2 = 80$, $t_2 = 35$. Hoca önce P1'in çalışma süresini 20 diye okudu, sonra slaytta 25'e çıkarıldığını fark edip düzeltti.

$$U = \frac{25}{50} + \frac{35}{80} = 0{,}5 + 0{,}4375 = 0{,}9375$$

Kullanım hâlâ 1'den küçüktür. Zamanlanabilir gibi görünür. Rate monotonic ile zamanlayınca:

- 0-25: P1 çalışır (periyodu daha kısa, önceliği yüksek).
- 25-50: P2 çalışır. 35 biriminin 25'ini tamamlar, 10 birimi kalır.
- 50: P1 yeniden gelir ve P2'yi keser. 50-75: P1 çalışır.

- 75-85: P2 kalan 10 birimi çalışır ve 85'te biter. Oysa ilk son tarihi 80'dir.

P2 son tarihini kaçırır. Kullanım 0,94 olsa da, yani 1'den küçük olsa da bu iş kurtarmaz. Sıra ve kaçırma Python simülasyonuyla doğrulandı.

> **Ders sonrası not:** Hoca "35 bölü 80, 0,94" dedi. $35/80 = 0{,}4375$'tir. 0,94 (tam olarak 0,9375), iki görevin toplam kullanımıdır.

9.3 Rate Monotonic İçin Kullanım Sınırı

Neden kurtarmadı? Hoca bunu bir formülle açıkladı. N süreç sayısı olmak üzere, rate monotonic ile en kötü durumda garanti edilebilen CPU kullanımı:

$$U_{\text{sınır}} = N\left(2^{1/N} - 1\right)$$

- $N = 1$: $1 \cdot (2^1 - 1) = 1$. Yüzde yüz kullanılabilir.
- $N = 2$: $2(\sqrt{2} - 1) \approx 0{,}828$. Hocanın deyişiyle aşağı yukarı %83 ile sınırlanır.
- Süreç sayısı arttıkça elde edilebilecek kullanım giderek düşer. Python ile: $N = 3$ için yaklaşık 0,780, $N = 10$ için yaklaşık 0,718.

Örnek 1'de kullanım 0,75'tir, iki görev için sınırın (0,828) altındadır, bu yüzden garantilidir. Örnek 2'de kullanım 0,9375'tir, sınırın üstündedir; garanti yoktur ve gerçekten bir son tarih kaçırılır (ara adım eklendi).

> **Ek bilgi (derste anlatılmadı):** Bu sınır Liu ve Layland'in 1973 tarihli çalışmasından gelir (tarih büyük olasılıkla bu). N sonsuza giderken $N(2^{1/N} - 1)$ ifadesi $\ln 2 \approx 0{,}693$'e yaklaşır. Sınır yeterli bir koşuldur, gerekli değildir: Kullanımı sınırın üstünde olan bir görev kümesi de bazen rate monotonic ile zamanlanabilir.

9.4 En Erken Son Tarih Önce

Hoca, derdimiz son tarihleri yakalamaksa farklı bir yerden yaklaşmamız gerektiğini söyledi: periyotlar üzerinden değil, son tarihler üzerinden. En erken son tarih önce (earliest deadline first, EDF) zamanlamada son tarihi en yakın olan görev en yüksek önceliği alır. Öncelik dinamiktir; her yeni iş geldiğinde son tarihlere göre yeniden belirlenir.

> **Ders sonrası not:** Hoca önce "deadline ne kadar uzaksa önceliği o kadar yüksek" dedi, hemen ardından "ne kadar uzaksa önceliği o kadar düşük" diye düzeltti. Doğrusu: Son tarihi ne kadar yakınsa öncelik o kadar yüksektir.

Örnek 2'deki görev kümesi (P1: $t = 25$, $p = 50$; P2: $t = 35$, $p = 80$) EDF ile şöyle çalışır. Python simülasyonuyla doğrulandı:

- 0-25: P1 çalışır. Son tarihi 50, P2'ninki 80'dir. P1'inki daha yakındır.
- 25-60: P2 çalışır. 50'de P1 yeniden gelir, ama yeni son tarihi 100'dür. P2'ninki (80) daha yakın olduğu için P2 kesilmez ve 60'ta biter.
- 60-85: P1 çalışır. 80'de P2 yeniden gelir, son tarihi 160'tır. P1'in son tarihi (100) hâlâ daha yakın olduğu için P1 devam eder, 85'te biter.
- 85-100: P2 çalışır, 35 biriminin 15'ini tamamlar.
- 100: P1 yeniden gelir, son tarihi 150'dir. P2'ninki 160 olduğu için P1 öne geçer. 100-125: P1.
- 125-145: P2 kalan 20 birimi çalışır ve 145'te biter (son tarih 160).

Bu şekilde iki görev de bütün son tarihlerini karşılayarak çalışır.

EDF teorik olarak optimaldir. Bir görev kümesinin her sürecin son tarihini karşılayacak biçimde zamanlanması mümkünse EDF bunu sağlar. Hocanın ifadesiyle: EDF bir sistemi zamanlayamıyorsa, rate monotonic'in zamanlamasını bekleyemezsiniz.

> **Ek bilgi (derste anlatılmadı):** EDF için zamanlanabilirlik koşulu, son tarihler periyoda eşitken basittir: Toplam kullanım $U \leq 1$ ise görev kümesi zamanlanabilir. Bu yüzden Örnek 2'deki 0,9375'lik küme EDF ile zamanlanır. EDF'nin bedeli, önceliklerin çalışma sırasında sürekli yeniden hesaplanmasıdır.

9.5 Orantılı Pay Zamanlama

Bir başka yapı orantılı pay (proportional share) zamanlamadır. Hoca bunu ağırlıklı kuyruk zamanlamaya (weighted queue scheduling) benzetti; doğrudan bir karşılığı olmayan, daha çok teorik bir yapı olarak tanımladı. Sistemin CPU zamanı (yalnız CPU zamanlama açısından bakıyoruz) T tane parçaya bölünür ve her sürece belli bir miktar dağıtılır. Ağırlıklı round robin gibi düşünülebilir.

Örnek: A, B, C diye üç süreç ve toplam 100 birim (pay) vardır. CPU zamanı mantıksal olarak 100 birime bölünmüştür. A'ya 50, B'ye 15, C'ye 20 pay verilir.

Bunlar toplanınca $50+15+20=85$ eder, 100'ün tamamını kaplamaz. Bu bilinçlidir: Herhangi bir anda sisteme başka süreçler girebilir ya da çıkabilir. Hoca burada gerçek zamanlı sistemlerde zamanlamanın çevrimdışı ya da çevrimiçi yapılabileceğini tekrar hatırlattı. Çevrimdışı yapıldığında bir simülasyonla paylar dağıtılır ve sistem statik olarak öyle kurulur. Çevrimiçi yapıldığında kararlar dinamik verilir. Dinamik yapıda öyle bir pay bırakılmalıdır ki fazladan bir şey çıktığında sistem hayatına devam edebilsin.

9.6 POSIX Gerçek Zamanlı Zamanlama

POSIX standardı gerçek zamanlı zamanlama için iki yöntem sunar: FIFO (`SCHED_FIFO`) ve round robin (`SCHED_RR`). Hoca bunların hangi fonksiyonlarla kullanıldığını şu an gereksiz bir ayrıntı olarak gördü ve atladı.

10. İşletim Sistemi Örnekleri: Linux ve Windows

10.1 Linux

Hocaya göre Linux'ta üzerine bir şeyler söylenebilecek üç farklı zamanlayıcı vardı:

- **Çekirdek 2.5 öncesi:** Standart bir UNIX zamanlama algoritması kullanılıyordu. Çok etkin değildi; yapı olarak daha çok toplu (batch) işleme uygundu.
- **Çekirdek 2.5 ile $O(1)$ zamanlayıcı:** Öncelik tabanlı, preemptive bir yapıdır. İki ayrı öncelik aralığı vardır: 0-99 gerçek zamanlı aralık, ardından nice değerlerinin eşlendiği zaman paylaşımlı aralık (CFS kısmında verilen ölçeğe göre 100-139). Önceliği yüksek olan daha fazla time quantum alır.
- **Completely Fair Scheduler (CFS):** $O(1)$ zamanlayıcı etkileşimli süreçlerde, yani sistemin tepki verebilirliği (responsiveness) açısından zayıf kaldı. Bu yüzden CFS ortaya atıldı.

Nice değeri: Adı nezaketten gelir. Belirlediğiniz nice değeri ne kadar yüksekse diğer süreçlerin çalışmasına o kadar imkân verir, nezaket gösterirsiniz. Nice değeri ne kadar düşükse öncelik o kadar yüksektir. -20 gibi bir değer verirseniz daha bencil davranırsınız. $+19$ verirseniz diğerlerinin çalışmasına öncelik tanırsınız. Ölçek şöyledir: Normal öncelik 120'dir, nice -20 verirseniz 100'e, $+19$ verirseniz 139'a karşılık gelir.

> **Ders sonrası not:** Hoca $O(1)$ zamanlayıcının güzel tarafının önceki algoritmaya kıyasla "lineer zamanda çalışması" olduğunu söyledi. Doğrusu: $O(1)$ zamanlayıcı, seçim kararını süreç sayısından bağımsız, sabit zamanda verir. Önceki zamanlayıcı lineer zamanlıydı. Hocanın asıl vurgusu ("çok hızlı bir zamanlama kararı") doğrudur.

CFS'de zamanlama sınıfları: Linux bir sunucu sisteminde de, bir masaüstünde de, bir mobil cihazda da çalışabilir. Bu ortamların her birinin farklı zamanlama ihtiyaçları vardır. Bu yüzden sistemde farklı zamanlama sınıfları (scheduling class) tanımlanabilir. Sınıfların bir önceliği vardır; ayrıca sınıf içindeki her sürecin de ayrı bir önceliği vardır. Varsayılan olarak iki sınıf hazır gelir: varsayılan (default) sınıf ve gerçek zamanlı sınıf. Gerçek zamanlı sınıftaki öncelik değerleri statiktir. Varsayılan sınıfta öncelik, sürecin nice değerine bağlı olarak değişir. Zamanlayıcı, en yüksek öncelikli sınıftaki en yüksek öncelikli süreci başlatır.

Sanal çalışma süresi: Öncelik olan her yapıda açlık olabilir. Bunu engellemek için her süreç için bir sanal çalışma süresi (virtual runtime, vruntime) tutulur. Görevler vruntime'a göre dengeli bir ağaçta tutulur: daha düşük vruntime'lılar solda, daha yüksekler sağda. Zamanlayıcı, açlığı engellemek için en düşük vruntime'a sahip süreci seçer.

Ek bilgi (derste anlatılmadı): CFS'nin kullandığı dengeli ağaç bir kırmızı-siyah ağaçtır (red-black tree). Ekleme ve silme $O(\log n)$ sürer; en küçük vruntime'lı düğüm (en soldaki) ayrıca tutulduğu için seçim hızlıdır. vruntime, sürecin gerçek çalışma süresinin nice değerine göre ağırlıklandırılmış hâlidir: Düşük öncelikli sürecin vruntime'ı daha hızlı artar.

10.2 Windows

Hocaya göre Windows zamanlayıcısı daha karmakarışık bir yapıdır. Zamanlayıcının adı bile genel tanımlara aykırı olarak dağıtıcıdır (dispatcher). Bir round robin yapısı vardır; arka tarafta da çok seviyeli geri beslemeli kuyruk vardır.

32 seviyeli bir öncelik yapısı vardır:

- 1-15: değişken (variable) sınıf.
- 16-31: gerçek zamanlı (real-time) sınıf.

Hangi thread'in çalıştırılacağı bu öncelik değerine göre belirlenir. Sistemde CPU'yu isteyen başka bir thread yoksa, herhangi bir kuyrukta bekleyen yoksa ya da bekleyenlerin hepsi düşük öncelikliyse, thread çalışmaya devam eder. Ne zamana kadar? Şunlardan biri olana kadar:

- Thread kendini bloklar.
- Kendisine verilen zaman dilimini (time slice) kullanır.
- Kuyruğa daha yüksek öncelikli bir thread gelir.

Hoca bu ayrıntılara çok fazla ihtiyaç olmadığını söyledi. Solaris kısmına hiç girmedi. Gerekçesi, Solaris'in pek kullanılmayan bir sistem olması ve Linux ile Windows'ta konuşulanların bir tekrarı gibi olmasıydı. Yine de bu kısmın kitaptan okunmasını istedi.

11. Algoritma Değerlendirme

Son olarak hoca hangi zamanlama algoritmasının nasıl seçileceğini konuştu. Bölümün başında konuşulan zamanlama ölçütleri burada devreye girer. Geliştirilen sistemin ne için olduğu önemlidir. Sistemin tepki vermesi bekleniyorsa yanıt süresi (response time) önemlidir. Son tarihlerin olduğu bir yapıda dönüş süresini (turnaround time) en aza indirmek isteyebiliriz. Etkileşimli bir sistemde bekleme süresini en aza indirmek isteriz. Ölçüt, sistemin ne yaptığıyla ilgilidir.

Değerlendirme için birkaç yöntem vardır:

- **Deterministik modelleme (deterministic modeling):** Derste Gantt şemalarıyla yapıldığı gibi sistem modellenir. Her algoritma çalıştırıldığında ölçütün (örneğin ortalama bekleme süresinin) ne olduğu hesaplanır ve en düşük olan seçilir. Hızlı ve kolaydır. Sıkıntısı, her sürecin sisteme ne zaman geleceği, ne kadar çalışacağı gibi bilgilerin tam olarak gerekmesidir. Bunları toplamak kolay değildir.
- **Kuyruk modelleri (queueing models):** Ağlardaki gibi bir kuyruk modeli kurulup hesap yapılır. Burada kuyruk modelinin yapısı önem kazanır.
- **Simülasyon:** Belli açılardan daha iyi sonuç verir ve farklı durumlar altında test yapmayı sağlar. Örneğin gerçek bir sistemi çalıştırıp izler (trace) toplanır, bu izlerle farklı simülasyon ortamlarında hangi değerlerin elde edildiğine bakılır.

Bununla 6. bölüm bitti.

Örnek Sorular

Soru 1 (derste sözlü, 00:12:56): Round robin, yapı olarak FCFS'ye benzer. FCFS'de olmayıp round robin'de olan temel özellik nedir?

> **Çözüm (derste verildi):** Preemption. FCFS'de bir süreç işi bitene kadar CPU'yu bırakmaz. RR'de ise süreç en fazla bir time quantum kadar çalışır, sonra kesilip kuyruğun sonuna konur. RR ayrıca zaman paylaşımlı, etkileşimli sistemler için tasarlanmıştır.

Soru 2 (derste sözlü, 00:17:52 ve 00:19:21): P1 = 24, P2 = 3, P3 = 3 örneğinde time quantum 4 yerine 1 olsaydı ne olurdu? Çok uzun seçilseydi (ör. 24) ne olurdu?

> **Çözüm (derste verildi, sayılar Python ile):** $q = 1$'de sıra P1, P2, P3, P1, P2, P3, ... diye gider; bağlam değişimi sayısı artar (29 dilim sınırı, bunların 9'u farklı süreçler arası). Ek yük büyür. $q = 24$'te P1 baştan sona çalışır, P2 ve P3 bekler; RR, FCFS'ye döner ve ortalama bekleme 17 olur. Bu yüzden quantum, bağlam değişimi süresine göre büyük, ama etkileşimi bozmayacak kadar küçük seçilmelidir.

Soru 3 (derste sözlü, 00:42:38): SMP'de ortak hazır kuyruğun senkronizasyon sorununu çözmek için her işlemciye ayrı kuyruk verilirse bütün sorunlar çözülür mü?

> **Çözüm (derste verildi):** Hayır. Ayrı kuyruklarda yük dengesizliği oluşabilir: Bir kuyruk dolu, diğeri boş kalabilir. Bu yüzden yük dengeleme (push ve pull migration) gerekir. Dengelerken de işlemci yakınlığı gözetilmelidir, çünkü süreci başka işlemciye taşımak önbellek avantajını yok eder.

Soru 4 (derste sözlü, 00:58:41): Gerçek zamanlı sistem nedir?

> **Çözüm (derste verildi):** Bir öğrenci "önceden belirlenmiş bir zaman içinde görevlerin gerçekleşmesini garantileyen sistem" dedi. Hoca bunu kısmen doğru buldu. Önceden belirlenmiş bir çizelge şart değildir; zamanlama çevrimdışı ya da çevrimiçi yapılabilir. Gerçek zamanlı sistem zamana duyarlı bir sistemdir; amacı görevlerin belirlenen süre içinde tamamlanmasıdır. Son tarih kaçınca ciddi sonuç doğuyorsa sert (ABS), doğmuyorsa yumuşak (uydu alıcısı) gerçek zamanlıdır.

Soru 5 (derste sözlü, 01:14:44): Rate monotonic zamanlama her zaman işe yarar mı?

> **Çözüm (derste verildi, simülasyon Python ile):** Hayır. Rate monotonic yalnız periyoda bakar, son tarihleri dikkate almaz. P1 ($p = 50$, $t = 25$) ve P2 ($p = 80$, $t = 35$) kümesinde toplam kullanım $0{,}9375 < 1$ olduğu hâlde P2 ilk son tarihi olan 80'i kaçırır ve 85'te biter. İki görev için garanti edilen kullanım sınırı $2(\sqrt{2} - 1) \approx 0{,}828$'dir. Aynı küme EDF ile hiç son tarih kaçırmadan zamanlanır.

Dersin Toparlaması

Bu derste CPU zamanlama bölümü tamamlandı. Round robin, FCFS'ye preemption ekleyerek etkileşimli sistemlerde beklemeyi azaltır; başarımı büyük ölçüde time quantum seçimine bağlıdır. Quantum çok küçükse bağlam değişimi ek yükü artar, çok büyükse algoritma FCFS'ye döner. Turnaround ile bekleme süresi farklı ölçütlerdir ve etkileşimli sistemde asıl bakılan bekleme süresidir. Çok seviyeli kuyruklar süreçleri gruplara ayırıp her gruba farklı algoritma uygular. Geri beslemeli sürümü ise uzun CPU kullanan süreçleri alt kuyruklara indirerek SJF'ye benzer bir davranış elde eder.

Ardından thread zamanlamasında PCS ve SCS ayrımı, çok işlemcili sistemlerde asimetrik ve simetrik yapı, işlemci yakınlığı, NUMA, yük dengeleme ve donanım thread'leri anlatıldı. Gerçek zamanlı kısımda sert ve yumuşak sistem ayrımı, kesme ve dağıtım gecikmeleri, rate monotonic (ve neden kullanım 1'in altındayken bile son tarih kaçırabildiği), EDF ve orantılı pay zamanlama görüldü. Son olarak Linux ($O(1)$ ve CFS) ve Windows zamanlayıcıları ile algoritma değerlendirme yöntemleri konuşuldu.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.